

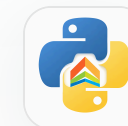


SCIPY 2026

MULTI-MODEL DATA IN PYTHON

ArcadeDB in Python: An In-Process Multi-Model Database

Taewoon Kim | HumemAI | taewoon.kim

`pip install arcadedb-embedded`

THE PROBLEM

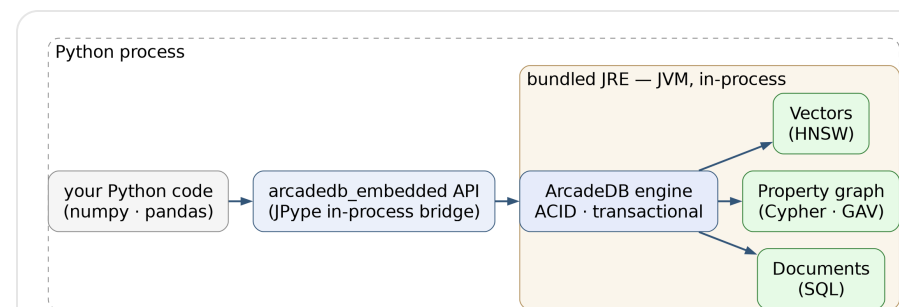
Scientific Python increasingly mixes **three** kinds of data access over the *same* data — records (SQL), relationships (graph), similarity over embeddings (vectors). Today that means **three separate systems + custom ETL + hand-kept consistency**.



THE IDEA · THE CONTRIBUTION

One embedded engine, in-process, one pip install

Documents, a property graph, and HNSW vectors in **one process, one file-backed database, one transaction** — no server. Engine = **ArcadeDB** (Apache-2.0 Java, credited); our contribution is the **Python binding**: JPytype in-process JVM + NumPy/pandas API + a **bundled per-platform JRE** → **20 wheels, no Java to install**.

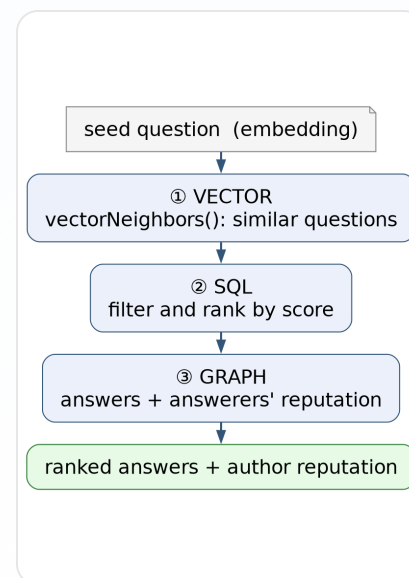


A Java engine sealed inside a Python wheel: one process hosts an in-process JVM (via JPytype) running ArcadeDB over a local file-backed DB; documents/SQL, property graph (Cypher) and HNSW vectors reached through **one handle**. Wheel ≈75 MB, plain `pip install`.

★ THE PAYOFF — ONE WORKFLOW, THREE MODELS

Hybrid retrieval: vector → SQL → graph, one transaction

Given a seed question, find semantically similar ones, keep the well-scored, return their best answers with each author's reputation. **Identifiers flow directly between steps — no serialization, no ETL, no second system.**



```
# 1) VECTOR — questions similar to a seed
cands = db.query("sql",
  "... expand(vectorNeighbors(?,?,?,?)) ...",
  "Question[embedding]", seed, 200, 100).to_list()

# 2) SQL — keep well-scored (ids from step 1)
filt = db.query("sql",
  f"... WHERE id IN {ids} AND score >= 1 ...").to_list()

# 3) GRAPH — MATCH to answers + reputation
hits = db.query("sql",
  f"... MATCH {{Question id IN {fids}}}
  -HAS_ANSWER->{{Answer}}-AUTHORED_BY->{{Userx}} ...").to_list()
```

≈130 ms warm end-to-end over all **213,761 questions + 208,986 answers. 1 process · 1 transaction · 0 ETL** (vector 81 · SQL 17 · graph 31 ms)

Capability matrix — only the multi-model engine covers all three in one process (✓ native ~ partial ✗ absent). A composable stack can reproduce it only with separate systems + glue.

CAPABILITY	SQLITE	DUCKDB	LADYBUG	CHROMA	ARCADE
Documents / OLTP SQL	✓	~	✗	✗	✓
Property graph + Cypher	✗	✗	✓	✗	✓
HNSW vector search	✗	✗	✓	✓	✓
All three in one engine	✗	✗	✗	✗	✓

HONEST RESULTS — VS SPECIALIST EMBEDDED PEERS

Fast where it's designed to be

24–31× Tabular OLTP throughput vs SQLite & DuckDB (≈5,355 vs ≈175–230 ops/s).

≈7× Graph OLTP throughput vs LadybugDB (≈3,480 vs ≈490 ops/s).

0.95 Vector recall@10 at matched HNSW (Chroma 0.97) — competitive *while* multi-model, with **lower peak memory** (17.3 vs 25.2 GiB, disk-backed index).

...AND WHERE IT ISN'T (WE DON'T HIDE IT)

Specialists win **heavy analytics**: DuckDB ≈9 ms vs ≈1,180 ms (tabular OLAP); LadybugDB ≈77 ms vs ≈844 ms (graph, a **GAV** narrows the gap). The standing cost is **memory** — a JVM baseline (≈930 MiB tabular) above the lean C peers; startup is sub-second, with an occasional GC tail. But these gaps are a **moving target**: ArcadeDB ships ~monthly and the binding packages each release, so Python benefits automatically.

USE IT WHEN

- ✓ local, reproducible, in-process work
- ✓ >1 model over the *same* data
- ✓ RAG / KB prototypes, teaching
- ✓ "one pip install" matters

REACH PAST IT WHEN

- one model dominates at scale
- warehouse columnar scans → DuckDB
- high-QPS vector serving
- hard real-time latency SLAs

CODE github.com/humemai/arcadedb-embedded-pythonDOCS docs.humem.ai/arcadedb

Apache-2.0

20 wheels · Py 3.10–3.14

matched-param benchmark

pinned versions + run manifest

datasets open

Reproducible end to end. Benchmarks: Cross Validated (stats.stackexchange.com) corpus, CC BY-SA; ArcadeDB 26.6.1, DuckDB 1.5.4, SQLite 3.46.1, LadybugDB 0.15.3, Chroma 1.5.9; 8-core cap. GenAI assisted language editing; all content reviewed & verified by the author.